VOYAGE ANALYTICS

How data becomes a working travel application

Read from top to bottom. Within each numbered row, follow the arrows from left to right.

A. PREPARE AND TRAIN | Build the models before users make predictions

1. Collect the data
Flights: journeys and prices
Users: age, company, labels
Hotels: stays and bookings

→

2. Check and prepare
Check columns and values.
Record the dataset version.
Split training / validation / test.

→

3. Train and compare
Predict flight prices.
Classify source gender labels.*
Rank hotel suggestions.

→

4. Evaluate and save
Choose using validation data.
Measure final test results.
Save model + preprocessing.

REUSABLE PYTHON CODE: src/voyage

The notebooks and Airflow call the same training functions, so the logic stays consistent.

Airflow: runs regression steps 2–4 in order

MLflow: records experiments and model versions

↓ Human review: select the approved model for release

B. PACKAGE AND DEPLOY | Turn the approved model into a running service

5. GitHub repository
Approved model files
Flask API + Streamlit code
Tests + deployment settings

→

6. Jenkins checks
Get the repository code.
Run automated tests.
Start the Docker image build.

→

7. DOCKER image
Package API code, libraries
and approved model together.
Test predictions in a container.

→

8. Local deployment
Jenkins runs the tested image.
A local Flask API is available.
This deployment was verified.

SEPARATE DEPLOYMENT PATHS | These are not automatic stages of the verified Jenkins run

Public demonstration
GitHub → Render builds/runs the Docker API.
GitHub → Streamlit Community Cloud runs the UI.
The public UI calls the public Render API.

Local Kubernetes demonstration
Load the API Docker image → run API replicas.
Verified: manually scale from 2 to 3 replicas.
Jenkins → Kubernetes and autoscaling: not verified.

↓ Once deployed, the application can answer requests

C. USE THE APPLICATION | What happens each time a user clicks Predict

9. User enters details
Journey details for a price,
or preferences for hotels.
Click the action button.

→

10. Streamlit screen
Collect the form inputs.
Send a request to Flask.
Wait for the result.

→

11. Flask API
Check the incoming inputs.
Use the saved model pipeline.
Create the prediction response.

→

12. Result on screen
Flask sends the result back.
Streamlit displays the estimate
or hotel recommendations.

Request → saved model → response. This flow does not train a new model.

Docker provides the API environment. The hosting service keeps the API running.

D. OBSERVE AND IMPROVE

API logs: errors, response time and model version. Review data changes and later real outcomes.

Human review → decide whether to retrain → repeat A → review and release through B.

* Classification is an educational source-label benchmark; it is not a reliable identity prediction.